

Lista básica de consultas esenciales en PostgreSQL

Usuarios (roles con LOGIN)

- **Usuarios o roles con LOGIN**

```
sql

SELECT rolname
FROM pg_roles
WHERE rolcanlogin = true;
```

- Todos los usuarios o roles

```
\du
```

- **Contar usuarios**

```
sql

SELECT COUNT(*) AS total_usuarios
FROM pg_roles
WHERE rolcanlogin = true;
```

Roles (todos los roles del sistema)

- **Listar roles**

```
sql

SELECT rolname
FROM pg_roles;
```

- **Contar roles**

```
sql

SELECT COUNT(*) AS total_roles
FROM pg_roles;
```

Roles asignados a un usuario

- **Roles de un usuario**

```
sql

SELECT r.rolname AS rol
FROM pg_auth_members am
JOIN pg_roles r ON am.roleid = r.oid
```

```
JOIN pg_roles u ON am.member = u.oid
WHERE u.rolname = 'maria';
```

Usuarios que pertenecen a un rol

- **Usuarios del rol vendedor**

sql

```
SELECT u.rolname AS usuario
FROM pg_auth_members am
JOIN pg_roles r ON am.roleid = r.oid
JOIN pg_roles u ON am.member = u.oid
WHERE r.rolname = 'vendedor';
```

- **Contar usuarios en un rol**

sql

```
SELECT COUNT(*) AS total
FROM pg_auth_members am
JOIN pg_roles r ON am.roleid = r.oid
WHERE r.rolname = 'vendedor';
```

- **Revocar un RLS o Rol de Grupo (Quitar herencia)**

```
REVOKE nombre_del_rol FROM nombre_usuario;
```

- Ejemplo real: Quitarle el rol de gerente a Sofía

```
REVOKE gerente FROM sofia_gerente;
```

Rol activo en este momento

- **Rol activo**

sql

```
SELECT session_user, current_user;
```

Esquemas

- **Listar esquemas**

sql

```
SELECT nsname AS esquema, pg_get_userbyid(nspowner) AS propietario
FROM pg_namespace
WHERE nsname NOT LIKE 'pg_%' AND nsname <> 'information_schema';
```

- \dn

- **Contar esquemas**

sql

```
SELECT COUNT(*) AS total_esquemas
FROM pg_namespace
WHERE nspname NOT LIKE 'pg_%' AND nspname <> 'information_schema';
```

Tablas por esquema

- **Listar tablas por esquema**

sql

```
SELECT tablename
FROM pg_tables
WHERE schemaname = 'ventas';
```

- \dt ventas.*;

- **Contar tablas por esquema**

sql

```
SELECT COUNT(*) AS total_tablas
FROM pg_tables
WHERE schemaname = 'ventas';
```

Secuencias por esquema

- **Listar secuencias**

sql

```
SELECT sequencename
FROM pg_sequences
WHERE schemaname = 'ventas';
```

- **Contar secuencias**

sql

```
SELECT COUNT(*) AS total_secuencias
FROM pg_sequences
WHERE schemaname = 'ventas';
```

Funciones por esquema

- **Listar funciones**

sql

```
SELECT proname
FROM pg_proc p
```

```
JOIN pg_namespace n ON p.pronamespace = n.oid
WHERE n.nspname = 'ventas';
```

- \df

- listar funciones de un esquema (ventas,stock ...etc)

```
\df *.stock
```

- ver el codigo de una funcion :

```
\sf stock.reducir_stock
```

- **Contar funciones**

```
sql
```

```
SELECT COUNT(*) AS total_funciones
FROM pg_proc p
JOIN pg_namespace n ON p.pronamespace = n.oid
WHERE n.nspname = 'ventas';
```

Permisos de una tabla

- **Ver permisos de una tabla**

```
sql
```

```
\z ventas.pedidos
```

Sí. En **PostgreSQL 18**, tienes varias formas de

ver los privilegios de un rol.

1. Ver los atributos del rol

En psql:

```
\du nombre_rol
```

Por ejemplo:

\du vioma

Te mostrará cosas como:

- Superuser
 - Create role
 - Create DB
 - Replication
 - Bypass RLS
-

2. Ver privilegios sobre tablas

Para una tabla concreta:

\dp nombre_tabla

Por ejemplo:

\dp productos

O mediante SQL:

```
SELECT *
FROM information_schema.role_table_grants
WHERE grantee = 'vioma';
```

Esto te permite ver privilegios como:

```
SELECT
INSERT
UPDATE
DELETE
```

3. Ver todos los privilegios de un rol

Una consulta bastante útil:

```
SELECT
    grantee,
    table_schema,
    table_name,
    privilege_type
FROM information_schema.role_table_grants
WHERE grantee = 'vioma'
ORDER BY table_schema, table_name, privilege_type;
```

4. Ver a qué roles pertenece

Esto es **muy importante**, porque un usuario puede recibir privilegios indirectamente mediante otro rol:

```
SELECT
    r.rolname AS rol,
    m.rolname AS miembro_de
FROM pg_auth_members am
JOIN pg_roles r ON r.oid = am.member
JOIN pg_roles m ON m.oid = am.roleid
WHERE r.rolname = 'vioma';
```

También puedes verlo directamente:

```
\du vioma
```

5. Ver privilegios sobre una base de datos

```
\l
```

O específicamente:

```
SELECT
    datname,
    datacl
FROM pg_database
WHERE datname = 'mi_basedatos';
```

Y para comprobar si el rol puede conectarse:

```
SELECT has_database_privilege(
    'vioma',
    'mi_basedatos',
    'CONNECT'
);
```

6. Comprobar exactamente qué puede hacer

Por ejemplo, si quieres saber si `vioma` puede hacer `SELECT`, `INSERT`, `UPDATE` y `DELETE` sobre `productos`:

```
SELECT
    has_table_privilege('vioma', 'productos', 'SELECT') AS puede_select,
    has_table_privilege('vioma', 'productos', 'INSERT') AS puede_insert,
    has_table_privilege('vioma', 'productos', 'UPDATE') AS puede_update,
    has_table_privilege('vioma', 'productos', 'DELETE') AS puede_delete;
```

Te devolverá algo como:

```
puede_select | puede_insert | puede_update | puede_delete
-----+-----+-----+-----
true         | true         | true         | false
```

Si estás administrando PostgreSQL, yo usaría principalmente `\du`, `\dp` y `has_table_privilege()`. Con esas tres puedes diagnosticar rápidamente quién es el rol, qué roles hereda y qué puede hacer realmente sobre los objetos.

Sí. Si quieres hacerlo **solo mediante SQL**, sin \du, puedes consultar pg_roles.

Para un rol concreto:

```
SELECT
    rolname,
    rolsuper,
    rolinherit,
    rolcreaterole,
    rolcreatedb,
    rolcanlogin,
    rolreplication,
    rolbypassrls
FROM pg_roles
WHERE rolname = 'vioma';
```

Por ejemplo, el resultado podría ser:

rolname	rolsuper	rolinherit	rolcreaterole	rolcreatedb	rolcanlogin	rolreplication	rolbypassrls
vioma	false	true	false	false	true		
false		false					

Qué significa cada atributo

Atributo	Significado
rolsuper	Es superusuario
rolinherit	Hereda automáticamente los privilegios de los roles de los que es miembro
rolcreaterole	Puede crear/modificar roles
rolcreatedb	Puede crear bases de datos
rolcanlogin	Puede iniciar sesión
rolreplication	Puede realizar conexiones de replicación
rolbypassrls	Puede saltarse Row-Level Security (RLS)

También puedes sacar **todos los atributos disponibles del rol**:

```
SELECT *
FROM pg_roles
WHERE rolname = 'vioma';
```

Y si quieres consultar **todos los roles del servidor**:

```
SELECT
    rolname,
    rolsuper,
    rolinherit,
    rolcreaterole,
    rolcreatedb,
    rolcanlogin,
```

```

rolreplication,
rolbypassrls
FROM pg_roles
ORDER BY rolname;

```

Si estás aprendiendo administración de PostgreSQL, `pg_roles` es una de las vistas del catálogo que merece la pena tener muy a mano.

y como sabemos si un rol puede hacer insert, select, delete etc?

Exacto. Aquí hay que distinguir **atributos del rol** (SUPERUSER, CREATEDB, etc.) de los **privilegios sobre objetos** (SELECT, INSERT, UPDATE, DELETE).

1. Ver los privilegios de un rol sobre una tabla

La forma más directa mediante SQL es:

```

SELECT
    grantee,
    table_schema,
    table_name,
    privilege_type
FROM information_schema.role_table_grants
WHERE grantee = 'vioma'
ORDER BY table_name, privilege_type;

```

Te podría devolver:

grantee	table_schema	table_name	privilege_type
vioma	public	productos	DELETE
vioma	public	productos	INSERT
vioma	public	productos	SELECT
vioma	public	productos	UPDATE

Por tanto, `vioma` puede hacer las cuatro operaciones sobre `productos`.

2. Comprobar una tabla concreta

También puedes preguntarle directamente a PostgreSQL:

```

SELECT
    has_table_privilege('vioma', 'public.productos', 'SELECT') AS select,
    has_table_privilege('vioma', 'public.productos', 'INSERT') AS insert,
    has_table_privilege('vioma', 'public.productos', 'UPDATE') AS update,
    has_table_privilege('vioma', 'public.productos', 'DELETE') AS delete;

```

Resultado:

select	insert	update	delete
true	true	true	false

Esto significa:

- SELECT
- INSERT
- UPDATE
- DELETE

Esta segunda forma es especialmente buena, porque `has_table_privilege()` tiene en cuenta los privilegios que el usuario obtiene **directamente, mediante otros roles y por otras reglas de PostgreSQL**.

3. Para todas las tablas de una base de datos

Puedes hacer algo muy práctico:

```
SELECT
    table_schema,
    table_name,
    has_table_privilege('vioma', table_schema || '.' || table_name, 'SELECT') AS
can_select,
    has_table_privilege('vioma', table_schema || '.' || table_name, 'INSERT') AS
can_insert,
    has_table_privilege('vioma', table_schema || '.' || table_name, 'UPDATE') AS
can_update,
    has_table_privilege('vioma', table_schema || '.' || table_name, 'DELETE') AS
can_delete
FROM information_schema.tables
WHERE table_schema NOT IN ('pg_catalog', 'information_schema')
ORDER BY table_schema, table_name;
```

Así tienes una especie de **matriz de permisos del rol**:

schema	table	can_select	can_insert	can_update	can_delete
public	productos	true	true	true	false
public	pedidos	true	false	false	false
public	usuarios	true	false	false	false

Para administración de PostgreSQL, yo me quedaría con esta idea:

pg_roles → qué es el rol.

pg_auth_members → de qué roles hereda.

has_*_privilege() → qué puede hacer realmente.

permisos de un usuario

La **forma completa y correcta** de ver **todos los permisos que tiene un usuario en PostgreSQL usando solo SQL** es consultar **tres niveles de privilegios**:

1. **Privilegios directos** (GRANT hechos al usuario)
2. **Privilegios heredados por roles** (roles asignados al usuario)

3. Privilegios por objeto (tablas, esquemas, funciones, secuencias)

Abajo tienes **todas las consultas SQL necesarias**, ya preparadas para copiar/pegar.

1) Ver los roles que hereda un usuario

Esto es clave porque **los permisos reales = permisos directos + permisos heredados**.

- **Roles heredados**

sql

```
SELECT
    r.rolname AS role,
    m.rolname AS member
FROM pg_auth_members am
JOIN pg_roles r ON am.roleid = r.oid
JOIN pg_roles m ON am.member = m.oid
WHERE m.rolname = 'jose';
```

Cambia 'jose' por tu usuario.

2) Ver todos los privilegios sobre tablas

PostgreSQL guarda los permisos en `information_schema.table_privileges`.

- **Permisos sobre tablas**

sql

```
SELECT
    grantee,
    table_schema,
    table_name,
    privilege_type
FROM information_schema.table_privileges
WHERE grantee = 'jose'
ORDER BY table_schema, table_name;
```

Esto te muestra: SELECT, INSERT, UPDATE, DELETE, REFERENCES, TRIGGER.

3) Ver permisos sobre esquemas

- **Permisos sobre esquemas**

sql

```
SELECT
    grantee,
    schema_name,
    privilege_type
FROM information_schema.schema_privileges
WHERE grantee = 'jose';
```

4) Ver permisos sobre secuencias

- **Permisos sobre secuencias**

sql

```
SELECT
    grantee,
    sequence_schema,
    sequence_name,
    privilege_type
FROM information_schema.sequence_privileges
WHERE grantee = 'jose';
```

5) Ver permisos sobre funciones

- **Permisos sobre funciones**

sql

```
SELECT
    grantee,
    routine_schema,
    routine_name,
    privilege_type
FROM information_schema.routine_privileges
WHERE grantee = 'jose';
```

6) Ver permisos sobre columnas

- **Permisos sobre columnas**

sql

```
SELECT
    grantee,
    table_schema,
    table_name,
    column_name,
    privilege_type
FROM information_schema.column_privileges
WHERE grantee = 'jose'
ORDER BY table_schema, table_name, column_name;
```

7) Ver ACLs crudas (la verdad absoluta)

Si quieres ver **todo lo que PostgreSQL guarda internamente**, usa `pg_class`:

- **ACL internas**

sql

```
SELECT
    n.nspname AS schema,
    c.relname AS object,
    c.relkind AS type,
```

```
c.relacl
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE c.relacl IS NOT NULL;
```

Aquí verás cosas como: {jose=arwdDxt/postgres} → permisos reales.

Resumen práctico

Si quieres **todos los permisos de un usuario**, ejecuta **estos tres**:

1. Roles heredados → `pg_auth_members`
2. Permisos por objeto → `information_schema.*_privileges`
3. ACL internas → `pg_class.relacl`

Con eso tienes **100% de los permisos**, sin psql, solo SQL.

Si quieres, puedo prepararte un **script único** que te genere un **informe completo** de permisos para cualquier usuario. ¿Quieres el informe para [jose] o para otro usuario?

En que se diferencia **information_schema.role_table_grants** de **information_schema.table_privileges**

La diferencia entre `information_schema.role_table_grants` y `information_schema.table_privileges` es **muy importante**, aunque a primera vista parecen casi iguales. La clave está en **qué permisos muestran** y **cómo los interpretan**.

Resumen rápido

- `role_table_grants` → muestra **solo los permisos otorgados mediante GRANT**, incluyendo permisos **heredados por roles**.
- `table_privileges` → muestra **solo los permisos directos del usuario**, sin expandir herencias.

Diferencia técnica y conceptual

1) `information_schema.table_privileges`

Muestra **privilegios explícitos** que se han dado directamente al usuario con:

```
sql
GRANT SELECT ON tabla TO jose;
```

Características:

- No expande herencias de roles.
- Solo muestra permisos directos.
- No incluye permisos por ser dueño de la tabla.
- No incluye permisos implícitos.

Ejemplo: si jose pertenece al rol vendedor, y vendedor tiene permisos sobre la tabla, **no aparecerán aquí.**

2) `information_schema.role_table_grants`

Muestra **todos los permisos que un rol tiene sobre tablas**, incluyendo:

- Permisos directos
- Permisos heredados por roles
- Permisos otorgados a roles que incluyen al usuario
- Permisos otorgados a PUBLIC
- Permisos otorgados a roles padres

Es decir: **expande la herencia.**

Esto es lo que realmente responde a la pregunta: *"¿Qué permisos tiene este usuario en total?"*

Ejemplo práctico

Supón:

sql

```
GRANT SELECT ON ventas.pedidos TO vendedor;
GRANT vendedor TO jose;
```

Entonces:

Resultado en `table_privileges`:

- **NO aparece nada para jose** (porque el GRANT fue al rol vendedor, no a jose)

Resultado en `role_table_grants`:

- **Sí aparece SELECT para jose** (porque hereda permisos del rol vendedor)

Diferencia clave en una frase

- `table_privileges` = permisos directos del usuario
- `role_table_grants` = permisos efectivos del usuario (incluye herencias)

Tabla comparativa

Vista	Muestra	Incluye herencia	Incluye permisos de PUBLIC	Útil para
<code>table_privileges</code>	GRANT directos	No	No	Ver permisos explícitos
<code>role_table_grants</code>	GRANT efectivos	Sí	Sí	Ver permisos reales del usuario

¿Cuál deberías usar para auditoría real?

Siempre `role_table_grants`, porque refleja **lo que el usuario realmente puede hacer**.

Permisos de usuarios y roles.

Da igual que pongas un usuario o un rol, porque PostgreSQL trata **usuarios y roles como la misma entidad interna**. La diferencia está en **cómo se interpretan los permisos**, no en cómo se consultan.

Idea clave

En PostgreSQL:

Un usuario = un rol con LOGIN Un rol = un grupo de permisos

Por eso, cuando consultas vistas como:

- `role_table_grants`
- `table_privileges`
- `schema_privileges`

Puedes poner **un usuario o un rol**, y PostgreSQL lo tratará igual.

Entonces... ¿dónde está la diferencia real?

1) Si consultas un usuario

Verás:

- Permisos directos del usuario
- Permisos heredados por roles
- Permisos otorgados a PUBLIC
- Permisos por ser dueño de objetos

2) Si consultas un rol

Verás:

- Permisos directos del rol
- Permisos heredados por roles padres
- Permisos otorgados a PUBLIC
- Permisos por ser dueño de objetos (si el rol es dueño)

Pero no verás permisos heredados por usuarios, porque los usuarios no “dan permisos” hacia arriba.

Ejemplo claro

Roles:

Código

```
GRANT SELECT ON ventas.pedidos TO vendedor;  
GRANT vendedor TO jose;
```

Si consultas permisos de jose:

- Verás SELECT (porque lo hereda de vendedor)

Si consultas permisos de vendedor:

- Verás SELECT (porque lo tiene directamente)

Si consultas permisos de otro usuario:

- No verás nada

Conclusión práctica

- **Sí**, puedes consultar permisos usando un usuario o un rol.
- **No cambia la consulta**, solo cambia **qué permisos aparecen**.
- Para ver permisos **efectivos**, usa siempre **role_table_grants** con el **usuario real**.

todas esas vistas de information_schema muestran únicamente permisos que fueron otorgados mediante GRANT. No muestran permisos “mágicos” ni implícitos: solo **GRANTs reales** registrados en las ACL internas.

Vamos punto por punto para que quede cristalino.

Resumen directo

Todas estas vistas:

- `table_privileges`
- `schema_privileges`
- `sequence_privileges`
- `column_privileges`

solo muestran permisos otorgados con GRANT.

Pero cada una tiene **dos matices importantes**:

1. **Solo muestran GRANTs directos**, no permisos heredados por roles.
2. **No muestran permisos por ser dueño del objeto** (ownership).

¿Qué significa “solo GRANTs directos”?

Ejemplo:

sql

```
GRANT SELECT ON ventas.pedidos TO vendedor;  
GRANT vendedor TO jose;
```

Si consultas `table_privileges` para jose:

- **NO aparece nada** (porque el GRANT fue al rol vendedor, no a jose)

Si consultas `table_privileges` para vendedor:

- **Sí aparece SELECT** (porque el GRANT fue directo al rol)

¿Entonces cómo veo permisos heredados?

Ahí entra:

- `role_table_grants`

Esta vista **sí expande herencias** y muestra permisos efectivos.

¿Y los permisos por ser dueño del objeto?

Ninguna vista de `information_schema` los muestra.

Ejemplo: si jose creó la tabla, tiene todos los permisos, pero:

- `table_privileges` → **no muestra nada**
- `role_table_grants` → **tampoco**

Para ver permisos por ownership necesitas mirar las ACL internas:

- **ACL internas**

sql

```
SELECT relname, relacl
FROM pg_class
WHERE relacl IS NOT NULL;
```

Tabla comparativa clara

Vista	Muestra	Herencia Ownership	Fuente
table_privileges	GRANT directos		GRANT
schema_privileges	GRANT directos		GRANT
sequence_privileges	GRANT directos		GRANT
column_privileges	GRANT directos		GRANT
role_table_grants	GRANT efectivos		GRANT + herencia
ACL internas	Todo (GRANT + ownership)		ACL

Conclusión

Sí: **todas esas vistas dependen exclusivamente de GRANTs**. No muestran:

- permisos heredados por roles
- permisos por ser dueño
- permisos implícitos del sistema

Para ver **todo**, necesitas combinar:

- `role_table_grants` → permisos efectivos
- `pg_class.relacl` → permisos reales incluyendo ownership

Si quieres, te preparo un **script único** que te muestre absolutamente todos los permisos de un usuario (directos, heredados, ownership, ACL). ¿Quieres el informe para **jose** o para otro rol?

pero con role_table_grants se pueden tambien ver tablas, esquemas columnas y secuencias?

La respuesta corta, Jose: **NO**, `information_schema.role_table_grants` **solo muestra permisos sobre TABLAS**. No muestra:

- permisos sobre **esquemas**
- permisos sobre **columnas**
- permisos sobre **secuencias**
- permisos sobre **funciones**

Y esto es *intencional*: PostgreSQL solo creó esa vista para **tablas**.

Vamos a verlo bien estructurado.

Qué muestra realmente `role_table_grants`

Solo muestra **GRANTs efectivos sobre tablas**, incluyendo:

- permisos directos
- permisos heredados por roles
- permisos otorgados a PUBLIC
- permisos otorgados a roles padres

Pero **solo para objetos de tipo tabla**.

Puedes profundizar en `role_table_grants`.

¿Por qué no muestra esquemas, columnas o secuencias?

Porque PostgreSQL **no tiene**:

- `role_schema_grants`
- `role_sequence_grants`
- `role_column_grants`

Es decir, **no existen vistas equivalentes que expandan herencias** para otros tipos de objetos.

Por eso:

- Para **tablas** → `role_table_grants`
- Para **todo lo demás** → solo existen vistas que muestran **GRANTs directos**, sin herencia:
 - `schema_privileges`
 - `sequence_privileges`
 - `column_privileges`
 - `routine_privileges`

Entonces... ¿cómo ver permisos efectivos (incluyendo herencias) en esquemas, columnas y secuencias?

No se puede con `information_schema`.

La única forma es:

1) Ver roles heredados

sql

```
SELECT r.rolname AS role
FROM pg_auth_members am
JOIN pg_roles r ON am.roleid = r.oid
WHERE am.member = (SELECT oid FROM pg_roles WHERE rolname = 'jose');
```

2) Ver GRANTs directos en las vistas de `information_schema`

3) Ver ACL internas en `pg_class` y `pg_namespace`

Ejemplo para secuencias:

sql

```
SELECT n.nspname, c.relname, c.relacl
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE c.relkind = 'S';
```

Esto sí muestra:

- permisos directos
- permisos heredados
- permisos por ownership
- permisos otorgados a PUBLIC

Resumen claro

Tipo de objeto	Vista con GRANTs directos	Vista con permisos efectivos (herencia)
Tablas	<code>table_privileges</code>	<code>role_table_grants</code>
Esquemas	<code>schema_privileges</code>	No existe
Secuencias	<code>sequence_privileges</code>	No existe
Columnas	<code>column_privileges</code>	No existe
Funciones	<code>routine_privileges</code>	No existe

Conclusión

- `role_table_grants` solo sirve para tablas.
- Para esquemas, secuencias, columnas y funciones **solo puedes ver GRANTs directos**.
- Para permisos efectivos (herencias + ownership) debes mirar **ACL internas**.

estoy como jose y quiero cambiarme al rol de juani

Para “cambiarte” al rol **juani**, Jose, hay dos formas posibles en PostgreSQL, y solo una funciona dependiendo de cómo hayas creado ese rol.

Aquí tienes la explicación directa y sin rodeos.

1) Si juani es un usuario con LOGIN

(Es decir, un rol que puede conectarse)

Entonces **NO puedes cambiarte a él dentro de la sesión**. PostgreSQL no permite “convertirte” en otro usuario con LOGIN.

En ese caso, solo puedes **reconectar**:

```
sql
```

```
\c tienda_electronica juani
```

o con contraseña:

```
sql
```

```
\c "dbname=tienda_electronica user=juani password=TU_PASSWORD"
```

2) Si juani es un rol sin LOGIN

(Es decir, un rol de permisos, como “vendedor”, “auditor”, etc.)

Entonces **sí puedes activarlo** dentro de tu sesión actual:

```
sql
```

```
SET ROLE juani;
```

Y para volver a tu usuario original:

```
sql
```

```
RESET ROLE;
```

¿Cómo saber si juani es usuario o rol?

Ejecuta:

```
sql
```

```
SELECT rolname, rolcanlogin  
FROM pg_roles  
WHERE rolname = 'juani';
```

- Sirolcanlogin = true → **es usuario**
- Sirolcanlogin = false → **es rol**

Resumen rápido

- Si **juani** es **usuario** → reconectar con \c
- Si **juani** es **rol** → activar con SET ROLE juani;

Comando para ver qué rol tienes activo ahora

sql

```
SELECT session_user, current_user;
```

- session_user → tú (jose)
- current_user → el rol activo (si hiciste SET ROLE)

POLITICAS.

- **ver las políticas actuales**

```
SELECT polycname, roles, cmd, permissive, qual, with_check
FROM pg_policies
WHERE schemaname = 'ventas'
AND tablename = 'pedidos';
```

. HACER QUE UN USUARIO (JUANI) PUEDA SER GERENTE Y VER TODOS LOS PEDIDOS.

Sql

```
SELECT polycname, roles, cmd, permissive, qual, with_check
FROM pg_policies
WHERE schemaname = 'ventas'
AND tablename = 'pedidos';
-----
```

```
"pedidos_gerente" "{gerente}" "SELECT" "RESTRICTIVE" "true"
"pedidos_por_usuario" "{public}" "SELECT" "PERMISSIVE" "(usuario =
CURRENT_USER)"
```

```
-- 1. Le quitamos el rol de cliente a Juani
REVOKE vendedor FROM juani;
```

```
-- 2. Le asignamos el nuevo rol de gerente
GRANT gerente TO juani;
```

1. El peligro del tipo **RESTRICTIVE** (Restringido)

Tienes la política de gerentes configurada como **RESTRICTIVE**, mientras que la de usuarios normales está como **PERMISSIVE**.

En PostgreSQL, las políticas funcionan así:

- Las políticas **PERMISSIVE** (Permisivas) se suman con un **OR**.
- Las políticas **RESTRICTIVE** (Restrictivas) se aplican después como un filtro obligatorio usando un **AND**.

Al poner la de gerente como **RESTRICTIVE**, le estás diciendo a PostgreSQL: *"No importa qué diga cualquier otra política, **obligatoriamente** se tiene que cumplir la condición de gerente"*. Como Juani es gerente, pasa ese filtro, pero luego PostgreSQL lo evalúa contra la política permisiva (`usuario = CURRENT_USER`). Como ambas se terminan cruzando de forma estricta por culpa del diseño, Juani queda atrapado viendo solo lo suyo.

2. La condición (`usuario = CURRENT_USER`)

Tu política permisiva dice exactamente `usuario = CURRENT_USER`. Si Juani está conectado como `juani`, el motor solo le va a devolver las filas donde la columna `usuario` sea exactamente igual a `'juani'`.

Cómo solucionarlo ahora mismo

Para que Juani (y cualquier gerente) pueda ver todos los pedidos, **todas las políticas de consulta (SELECT) deben ser PERMISSIVE**. De esa forma, PostgreSQL dirá: *"O ves tus propios pedidos OR eres gerente y ves todo"*.

Ejecuta este código como administrador para borrar esas políticas y crearlas correctamente:

SQL

```
-- 1. Borramos las políticas problemáticas que tienes actualmente
DROP POLICY IF EXISTS pedidos_gerente ON ventas.pedidos;
DROP POLICY IF EXISTS pedidos_por_usuario ON ventas.pedidos;

-- 2. Creamos la política para usuarios normales (Permisiva)
CREATE POLICY pedidos_por_usuario
ON ventas.pedidos
AS PERMISSIVE
FOR SELECT
TO public
USING (usuario = CURRENT_USER);

-- 3. Creamos la política para gerentes (¡También Permisiva!)
CREATE POLICY pedidos_gerente
ON ventas.pedidos
AS PERMISSIVE
FOR SELECT
TO gerente
USING (true); -- "true" significa que el rol gerente tiene permitido ver TODAS
las filas
```

```

SELECT polycyname, roles, cmd, permissive, qual, with_check
FROM pg_policies
WHERE schemaname = 'ventas'
    AND tablename = 'pedidos';
-----
"pedidos_por_usuario"    "{public}"    "SELECT"    "PERMISSIVE" "(usuario =
CURRENT_USER)"

"pedidos_gerente" "{gerente}" "SELECT"    "PERMISSIVE"    "true"

```

¿Por qué esto sí va a funcionar?

Al ser ambas políticas **PERMISSIVE**, PostgreSQL las unirá con un **OR**.

Cuando Juani consulte la tabla:

1. ¿El pedido es de Juani? (Política 1).
2. ¿Juani pertenece al rol gerente? **Sí**, por lo tanto la Política 2 devuelve **true** para **todas** las filas de la tabla.
3. Como Falso OR True = True, ¡Juani podrá ver el listado completo de pedidos automáticamente!

TRIGERS-

. ver todos los triggers

```

SELECT
    tname AS nombre_trigger,
    relname AS tabla_asociada,
    pg_get_triggerdef(pg_trigger.oid) AS
codigo_completo_del_trigger
FROM pg_trigger
JOIN pg_class ON pg_trigger.tgrelid = pg_class.oid
JOIN pg_namespace ON pg_class.relnamespace = pg_namespace.oid
WHERE tgisinternal = false
    AND nspname NOT IN ('pg_catalog', 'information_schema');

```

```

-----
\dy

```

. ver triggers de una tabla:

```
\d ventas.pedidos
```

. Script para ver qué ver el trigger junto con su función

```
SELECT
    tbl.relname AS tabla,
    trg.tgname AS nombre_trigger,
    -- Aquí vemos la definición del disparador (CREATE TRIGGER...)
    pg_get_triggerdef(trg.oid) AS definicion_del_trigger,
    -- Aquí vemos el código interno de la función que ejecuta
    p.prosrc AS codigo_interno_funcion
FROM pg_trigger trg
JOIN pg_class tbl ON trg.tgrelid = tbl.oid
JOIN pg_proc p ON trg.tgfoid = p.oid
WHERE trg.tgisinternal = false;
```